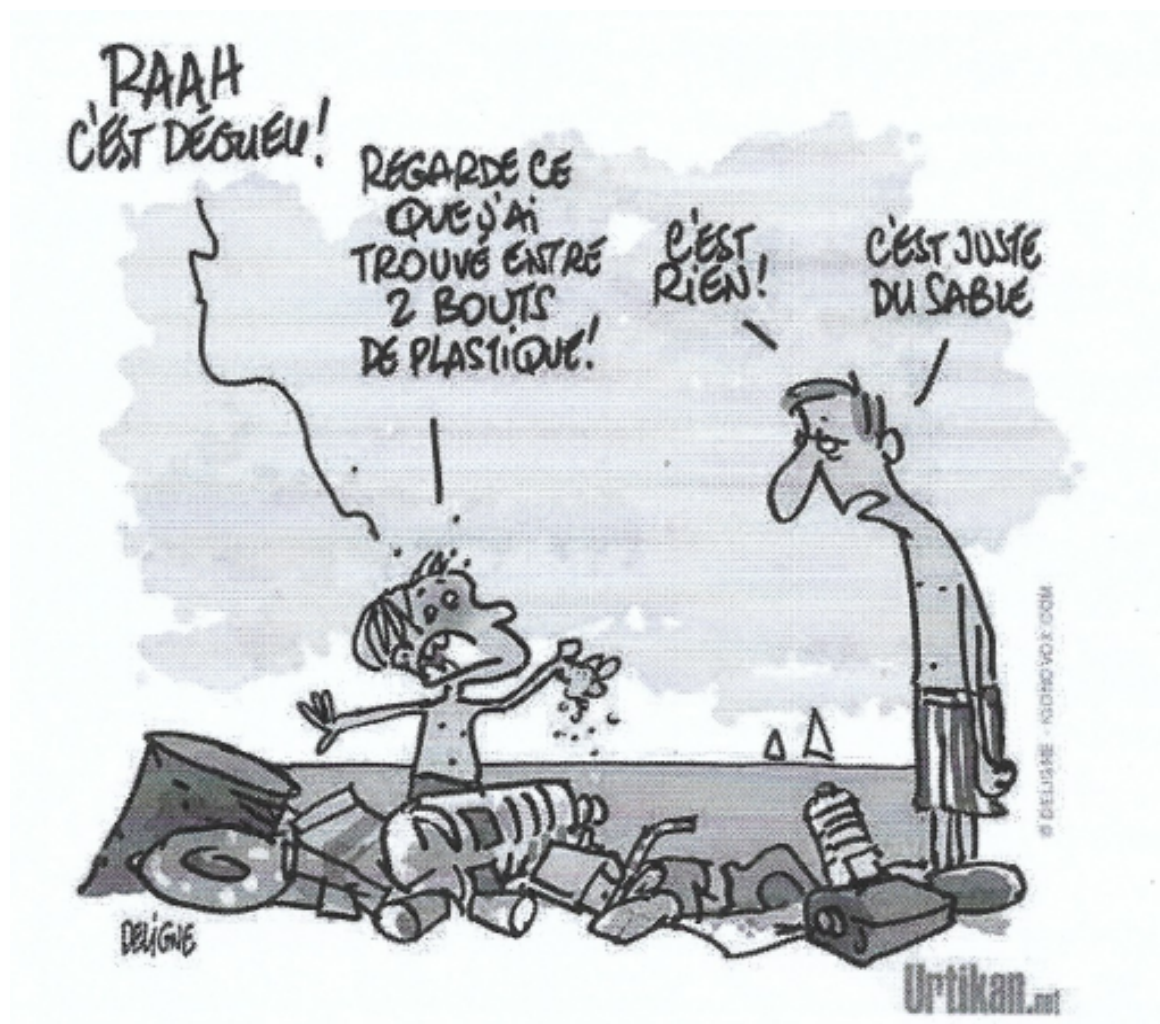


SAE S104 : BASE DES DONNÉES

Création base de données : Rapportage de la saison balnéaire 2024

Sommaire

2.1 : Script Manuel de création de la base de données.....	1
2.2 : Modélisation du script de creation avec PgModeler.....	2
2.3 Peuplement des tables.....	7



2.1 : Script Manuel de création de la base de données

```
-----  
  
-- Suppression des tables si elles existent  
DROP TABLE IF EXISTS analyses CASCADE;  
DROP TABLE IF EXISTS evenements CASCADE;  
DROP TABLE IF EXISTS sites CASCADE;  
DROP TABLE IF EXISTS communes CASCADE;  
DROP TABLE IF EXISTS departements CASCADE;  
DROP TABLE IF EXISTS regions CASCADE;  
  
--  
=====
```

-- Table REGIONS

```
--  
=====
```

CREATE TABLE regions (
 code VARCHAR PRIMARY KEY,
 nom VARCHAR NOT NULL
);

```
--  
=====
```

-- Table DEPARTEMENTS

```
--  
=====
```

CREATE TABLE departements (
 code VARCHAR PRIMARY KEY,
 region VARCHAR NOT NULL REFERENCES regions(code),
 nom VARCHAR NOT NULL
);

```
--  
=====
```

-- Table COMMUNES

```
--  
=====
```

CREATE TABLE communes (
 code VARCHAR PRIMARY KEY,
 departement VARCHAR NOT NULL REFERENCES departements(code),
 nom VARCHAR NOT NULL
);

```
--  
=====
```

-- Table SITES

```
--  
=====
```

CREATE TABLE sites (
 "idSite" VARCHAR PRIMARY KEY,
 nom VARCHAR NOT NULL,
 "codeCommune" VARCHAR NOT NULL REFERENCES communes(code),
 "dateDeclaration" DATE,
 "typeEau" VARCHAR,
 longitude REAL,
 latitude REAL

```

);

--
=====
-- Table EVENEMENTS
--
=====
CREATE TABLE evenements (
    "idEvenement" INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "idSite" VARCHAR NOT NULL REFERENCES sites("idSite"),
    evenement VARCHAR,
    debut DATE,
    fin DATE,
    mesure TEXT
);

--
=====
-- Table ANALYSES
--
=====

CREATE TABLE analyses (
    "idAnalyse" INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    "idSite" VARCHAR NOT NULL REFERENCES sites("idSite"),
    "datePrelevement" DATE NOT NULL,
    enterocoques INTEGER,
    escherichia INTEGER
);

--
=====
-- Vérification de la création des tables
--
=====
-----

```

2.2 : Modélisation du script de creation avec PgModeler

1/ Illustration comparatives cour/AGL commentée d'une association fonctionnelle:

IMAGE 1 : COUR

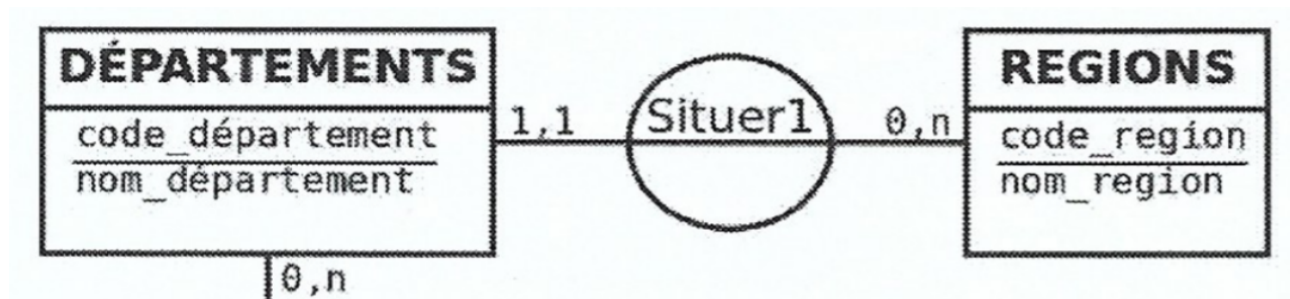
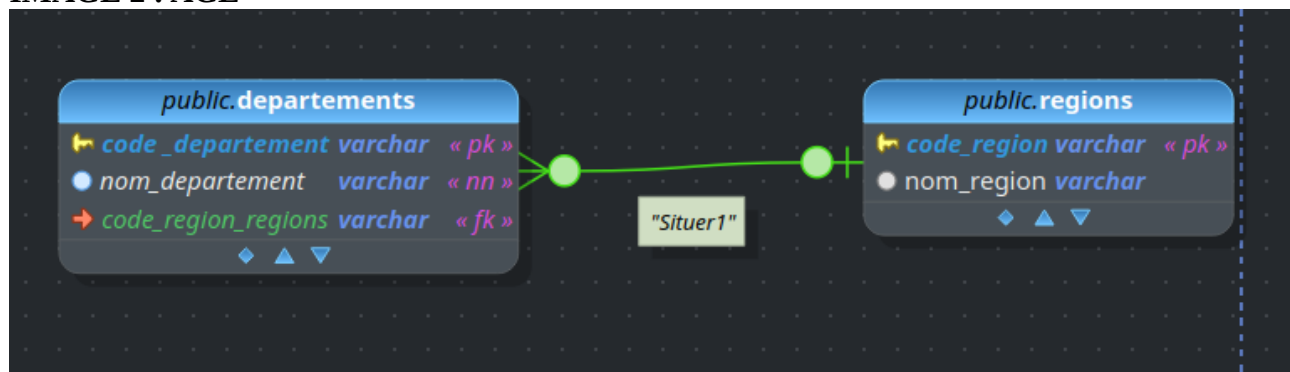


IMAGE 2 : AGL



Description comparative :

Dans le modèle théorique du cours, l'association est représentée par une bulle reliant les deux entités avec des cardinalités **(1,1)** et **(0,n)**. Cela indique qu'un département ne peut avoir qu'une seule région.

Dans le modèle du logiciel (AGL), cette association disparaît visuellement. Elle est remplacée par une **migration de clé** : l'identifiant de la table Régions est ajouté dans la table Départements. C'est ce qu'on appelle une **Clé Étrangère** (notée **FK**). C'est le moyen technique pour lier chaque département à sa région unique.

2/ Illustration comparatives cour/AGL commentée d'une association mailler:

IMAGE 1 : COUR

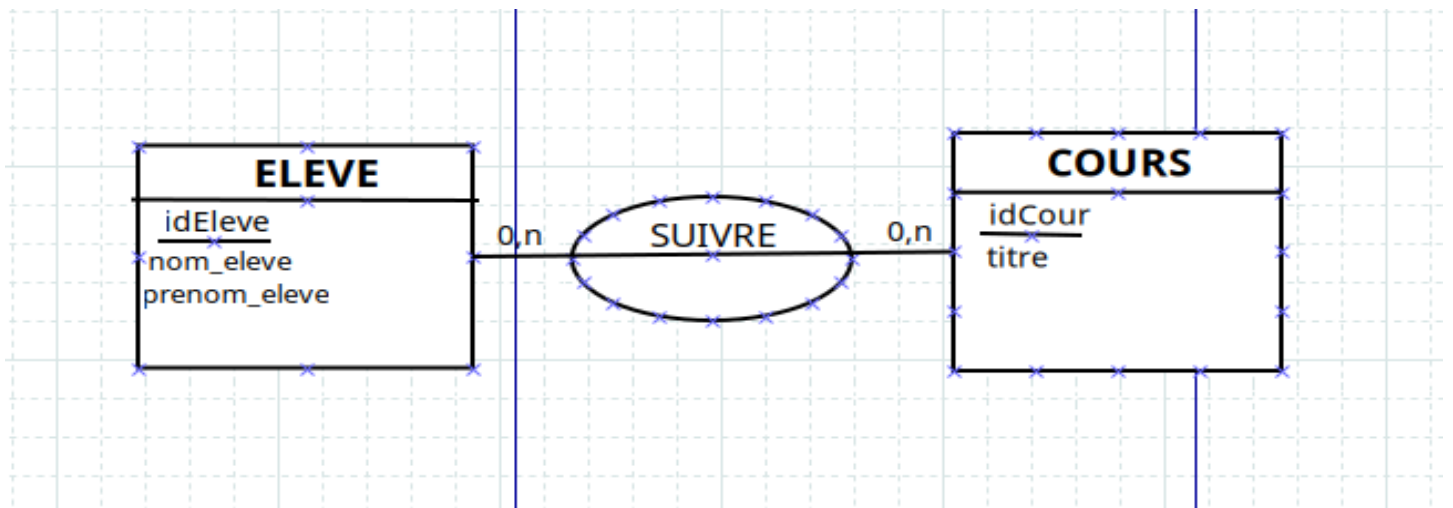
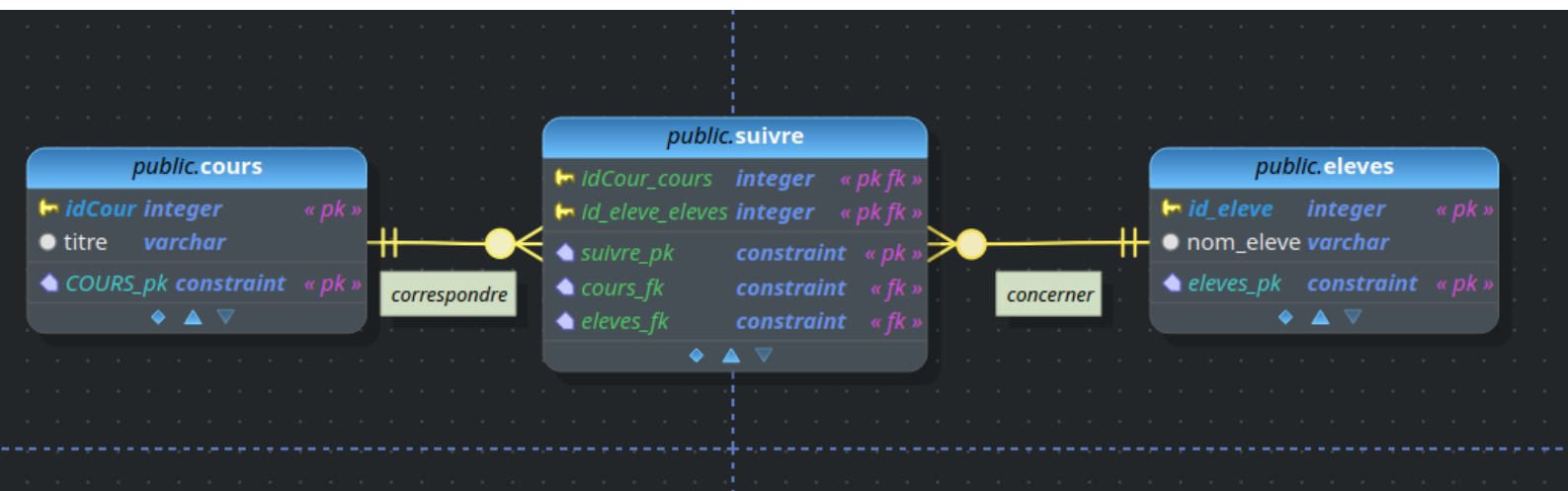


IMAGE 2 : AGL

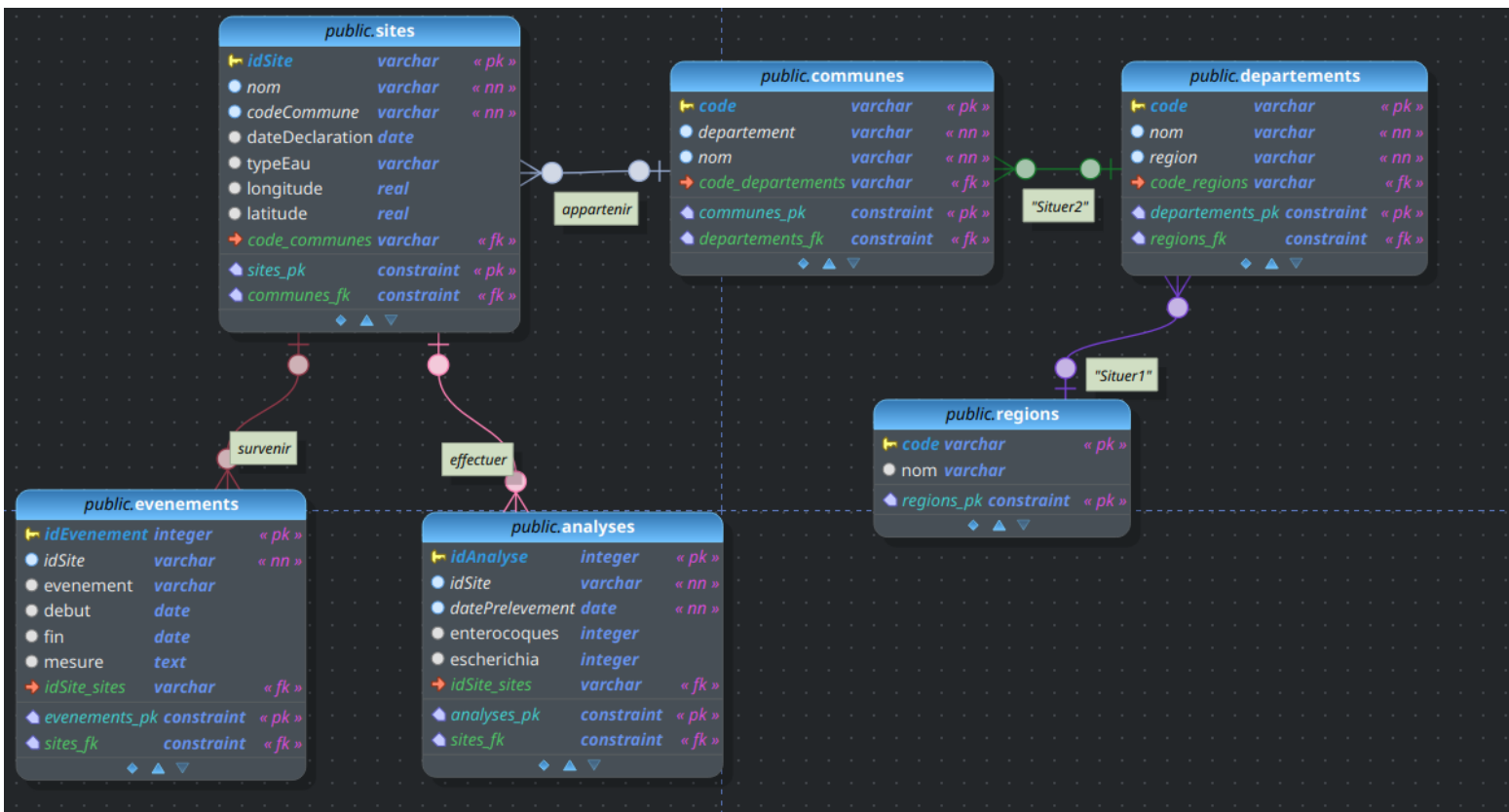


Description comparative :

Dans le modèle théorique du cours, l'association est représentée par une bulle avec des cardinalités **(0,n)** de chaque côté. Cela signifie qu'un élève peut suivre plusieurs cours et qu'un cours accueille plusieurs élèves.

Dans le modèle du logiciel (AGL), il est impossible de représenter cela par une simple clé étrangère. L'association se transforme donc en une **table de jointure** supplémentaire, ici nommée `public.suivre`. Cette table contient deux clés étrangères (`id_eleve_eleves` et `idCour_cours`) qui font référence aux clés primaires des deux tables d'origine. C'est cette table intermédiaire qui permet de stocker toutes les combinaisons possibles entre élèves et cours.

3/ Model physique de données réalisé avec l'AGL (PgModeler):



4/ Script SQL de creation des tables généré automatiquement par l'AGL :

```
-- Database generated with pgModeler (PostgreSQL Database Modeler).
-- pgModeler version: 1.1.0-beta1
-- PostgreSQL version: 16.0
-- Project Site: pgmodeler.io
-- Model Author: ---

-- Database creation must be performed outside a multi lined SQL file.
-- These commands were put in this file only as a convenience.
--
-- object: new_database | type: DATABASE --
-- DROP DATABASE IF EXISTS new_database;
CREATE DATABASE new_database;
-- ddl-end --

-- object: public.regions | type: TABLE --
-- DROP TABLE IF EXISTS public.regions CASCADE;
CREATE TABLE public.regions (
    code varchar NOT NULL,
    nom varchar,
    CONSTRAINT regions_pk PRIMARY KEY (code)
);
-- ddl-end --
ALTER TABLE public.regions OWNER TO postgres;
-- ddl-end --
```

```

-- object: public.departements | type: TABLE --
-- DROP TABLE IF EXISTS public.departements CASCADE;
CREATE TABLE public.departements (
    code varchar NOT NULL,
    nom varchar NOT NULL,
    region varchar NOT NULL,
    code_regions varchar,
    CONSTRAINT departements_pk PRIMARY KEY (code)
);
-- ddl-end --
ALTER TABLE public.departements OWNER TO postgres;
-- ddl-end --

-- object: regions_fk | type: CONSTRAINT --
-- ALTER TABLE public.departements DROP CONSTRAINT IF EXISTS regions_fk
CASCADE;
ALTER TABLE public.departements ADD CONSTRAINT regions_fk FOREIGN KEY
(code_regions)
REFERENCES public.regions (code) MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: public.communes | type: TABLE --
-- DROP TABLE IF EXISTS public.communes CASCADE;
CREATE TABLE public.communes (
    code varchar NOT NULL,
    departement varchar NOT NULL,
    nom varchar NOT NULL,
    code_departements varchar,
    CONSTRAINT communes_pk PRIMARY KEY (code)
);
-- ddl-end --
ALTER TABLE public.communes OWNER TO postgres;
-- ddl-end --

-- object: public.sites | type: TABLE --
-- DROP TABLE IF EXISTS public.sites CASCADE;
CREATE TABLE public.sites (
    "idSite" varchar NOT NULL,
    nom varchar NOT NULL,
    "codeCommune" varchar NOT NULL,
    "dateDeclaration" date,
    "typeEau" varchar,
    longitude real,
    latitude real,
    code_communes varchar,
    CONSTRAINT sites_pk PRIMARY KEY ("idSite")
);
-- ddl-end --
ALTER TABLE public.sites OWNER TO postgres;
-- ddl-end --

-- object: public.evenements | type: TABLE --
-- DROP TABLE IF EXISTS public.evenements CASCADE;
CREATE TABLE public.evenements (
    "idEvenement" integer NOT NULL GENERATED ALWAYS AS IDENTITY ,
    "idSite" varchar NOT NULL,

```

```

        evenement varchar,
        debut date,
        fin date,
        mesure text,
        "idSite_sites" varchar,
        CONSTRAINT evenements_pk PRIMARY KEY ("idEvenement")
    );
-- ddl-end --
ALTER TABLE public.evenements OWNER TO postgres;
-- ddl-end --

-- object: public.analyses | type: TABLE --
-- DROP TABLE IF EXISTS public.analyses CASCADE;
CREATE TABLE public.analyses (
    "idAnalyse" integer NOT NULL GENERATED ALWAYS AS IDENTITY ,
    "idSite" varchar NOT NULL,
    "datePrelevement" date NOT NULL,
    enterocoques integer,
    escherichia integer,
    "idSite_sites" varchar,
    CONSTRAINT analyses_pk PRIMARY KEY ("idAnalyse")
);
-- ddl-end --
ALTER TABLE public.analyses OWNER TO postgres;
-- ddl-end --

-- object: departements_fk | type: CONSTRAINT --
-- ALTER TABLE public.communes DROP CONSTRAINT IF EXISTS departements_fk
CASCADE;
ALTER TABLE public.communes ADD CONSTRAINT departements_fk FOREIGN KEY
(code_departements)
REFERENCES public.departements (code) MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: communes_fk | type: CONSTRAINT --
-- ALTER TABLE public.sites DROP CONSTRAINT IF EXISTS communes_fk CASCADE;
ALTER TABLE public.sites ADD CONSTRAINT communes_fk FOREIGN KEY
(code_communes)
REFERENCES public.communes (code) MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: sites_fk | type: CONSTRAINT --
-- ALTER TABLE public.evenements DROP CONSTRAINT IF EXISTS sites_fk CASCADE;
ALTER TABLE public.evenements ADD CONSTRAINT sites_fk FOREIGN KEY
("idSite_sites")
REFERENCES public.sites ("idSite") MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

-- object: sites_fk | type: CONSTRAINT --
-- ALTER TABLE public.analyses DROP CONSTRAINT IF EXISTS sites_fk CASCADE;
ALTER TABLE public.analyses ADD CONSTRAINT sites_fk FOREIGN KEY
("idSite_sites")
REFERENCES public.sites ("idSite") MATCH FULL
ON DELETE SET NULL ON UPDATE CASCADE;
-- ddl-end --

```

5 /Comparaison entre le script manuel et le script généré par l'AGL:

Premièrement, le script manuelle se distingue par une structure plus simple et lisible, avec des noms de tables et de colonnes raccourcis et harmonisés (par exemple, `code_region` devient `code`, `code_commune_communes` devient `codeCommune`), ce qui facilite la compréhension du modèle.

Ensuite, les relations entre les entités sont gérées via des **foreign keys**, définies directement dans les tables (par exemple, `commune.departement` fait référence à `departement.code`), contrairement au script AGL où elles sont ajoutées après coup via `ALTER TABLE`. Ces clés sont majoritairement `NOT NULL`, assurant une **intégrité référentielle stricte**. De plus, les types de données ont été adaptés à l'usage réel, comme `REAL` pour les coordonnées ou `TEXT` pour `mesure`.

Enfin, le script se concentre uniquement sur la définition du schéma relationnel, sans inclure la création de la base ni la gestion des propriétaires, ce qui le rend plus clair et maintenable. **Par exemple**, nous avons créé directement les tables `regions`, `departements`, `communes`, `sites`, `evenements` et `analyses` avec leurs clés primaires et les relations via des **foreign keys** (`departements.region REFERENCES regions(code)`, `communes.departement REFERENCES departements(code)`), sans commandes `CREATE DATABASE` ni `ALTER OWNER`.

2.3 Peuplement des tables

```
-- =====
-- SCRIPT DE PEUPLEMENT - SAISON BALNÉAIRE 2024
-- =====

-- =====
-- 1) CRÉATION DES TABLES TEMPORAIRES
-- =====

-- Création table temporaire régions / départements
DROP TABLE IF EXISTS tmp_departements;
CREATE TEMP TABLE tmp_departements (
    code_departement VARCHAR,
    nom_departement VARCHAR,
    code_region VARCHAR,
    nom_region VARCHAR
);

-- Création table temporaire sites et communes
DROP TABLE IF EXISTS tmp_sites;
CREATE TEMP TABLE tmp_sites (
    saison VARCHAR,
    region VARCHAR,
    departement VARCHAR,
    "idSite" VARCHAR,
```

```

    ancienCode VARCHAR,
    evolution VARCHAR,
    "nomSite" VARCHAR,
    "codeCommune" VARCHAR,
    nomCommune VARCHAR,
    "dateDeclaration" VARCHAR,
    "typeEau" VARCHAR,
    longitude VARCHAR,
    latitude VARCHAR
);

-- Création table temporaire événements
DROP TABLE IF EXISTS tmp_evenements;
CREATE TEMP TABLE tmp_evenements (
    saison VARCHAR,
    region VARCHAR,
    departement VARCHAR,
    "idSite" VARCHAR,
    typeEvenement VARCHAR,
    dateDebut VARCHAR,
    dateFin VARCHAR,
    mesure TEXT
);

-- Création table temporaire analyses
DROP TABLE IF EXISTS tmp_analyses;
CREATE TEMP TABLE tmp_analyses (
    saison VARCHAR,
    region VARCHAR,
    departement VARCHAR,
    "idSite" VARCHAR,
    "datePrelevement" VARCHAR,
    enterocoques VARCHAR,
    escherichia VARCHAR,
    statut VARCHAR,
    c1 VARCHAR,
    c2 VARCHAR,
    c3 VARCHAR
);

-- =====
-- 2) CHARGEMENT DES FICHIERS CSV
-- =====

-- Chargement CSV régions / départements
COPY tmp_departements
FROM '/tmp/departements-france.csv'
DELIMITER ',' CSV HEADER ENCODING 'UTF8';

-- Chargement CSV sites de baignade
COPY tmp_sites
FROM '/tmp/liste-des-sites-de-baignade-saison-balneaire-2024.csv'
DELIMITER ';' CSV HEADER ENCODING 'LATIN1';

-- Chargement CSV événements
COPY tmp_evenements
FROM '/tmp/saison-balneaire-2024-informations-sur-la-saison.csv'

```

```

DELIMITER ';' CSV HEADER ENCODING 'LATIN1';

-- Chargement CSV analyses de l'eau
COPY tmp_analyses
FROM '/tmp/saison-balneaire-2024-resultats-danalyses.csv'
DELIMITER ';' CSV HEADER ENCODING 'LATIN1';

-- =====
-- 3) PEUPLEMENT DES TABLES FINALES
-- =====

-- Insertion des régions
INSERT INTO regions(code, nom)
SELECT DISTINCT code_region, nom_region
FROM tmp_departements
ON CONFLICT (code) DO NOTHING;

-- Insertion des départements
INSERT INTO departements(code, region, nom)
SELECT DISTINCT code_departement, code_region, nom_departement
FROM tmp_departements
ON CONFLICT (code) DO NOTHING;

-- Insertion des communes
INSERT INTO communes(code, departement, nom)
SELECT DISTINCT
    "codeCommune",
    TRIM(LEADING '0' FROM departement),
    nomCommune
FROM tmp_sites
WHERE "codeCommune" IS NOT NULL
    AND TRIM(LEADING '0' FROM departement) IN (SELECT code FROM departements)
ON CONFLICT (code) DO NOTHING;

-- Insertion des sites de baignade
INSERT INTO sites("idSite", nom, "codeCommune", "dateDeclaration",
"typeEau", longitude, latitude)
SELECT DISTINCT
    "idSite",
    "nomSite",
    "codeCommune",
    TO_DATE("dateDeclaration", 'DD/MM/YYYY'),
    "typeEau",
    REPLACE(longitude, ',', '. '::REAL,
    REPLACE(latitude, ',', '. '::REAL
FROM tmp_sites
WHERE "idSite" IS NOT NULL
    AND "codeCommune" IN (SELECT code FROM communes)
ON CONFLICT ("idSite") DO NOTHING;

-- Insertion des événements
INSERT INTO evenements("idSite", evenement, debut, fin, mesure)
SELECT
    "idSite",
    typeEvenement,
    TO_DATE(dateDebut, 'DD/MM/YYYY'),
    TO_DATE(dateFin, 'DD/MM/YYYY'),

```

```

        mesure
FROM tmp_evenements
WHERE "idSite" IN (SELECT "idSite" FROM sites);

-- Insertion des analyses
INSERT INTO analyses("idSite", "datePrelevement", enterocoques, escherichia)
SELECT
    "idSite",
    TO_DATE("datePrelevement", 'DD/MM/YYYY'),
    NULLIF(enterocoques, '')::INTEGER,
    NULLIF(escherichia, '')::INTEGER
FROM tmp_analyses
WHERE "idSite" IN (SELECT "idSite" FROM sites);

-- =====
-- 4) SUPPRESSION DES TABLES TEMPORAIRES
-- =====

-- Suppression table temporaire régions / départements
DROP TABLE IF EXISTS tmp_departenements;

-- Suppression table temporaire sites
DROP TABLE IF EXISTS tmp_sites;

-- Suppression table temporaire événements
DROP TABLE IF EXISTS tmp_evenements;

-- Suppression table temporaire analyses
DROP TABLE IF EXISTS tmp_analyses;

```

Description commentée des étapes du script :

Pour garantir l'intégrité de la base, nous avons opté pour une approche par tables temporaires et opter pour un peuplement en 4 étapes.

1/Les fichiers CSV sont d'abord importés dans des tables de transit où toutes les colonnes sont typées en texte, ce qui évite les erreurs de formatage.

2/Les données sont ensuite nettoyées directement via SQL (conversion des dates, remplacement des virgules par des points pour les coordonnées GPS, transformation des champs vides en NULL)

3/Elles sont enfin insérées dans les tables définitives en respectant l'ordre hiérarchique des clés étrangères.

Nous avons d'ailleurs forcé l'encodage **LATIN1** lors de l'importation car les fichiers sources gouvernementaux utilisent historiquement cette norme et non l'UTF-8, (pour préserver les accents).

4/Enfin les tables temporaire ont été supprimées